

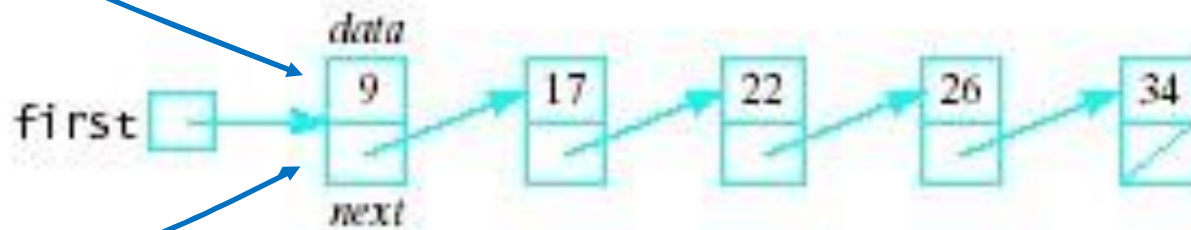
DATA STRUCTURES

Linked List



Linked List

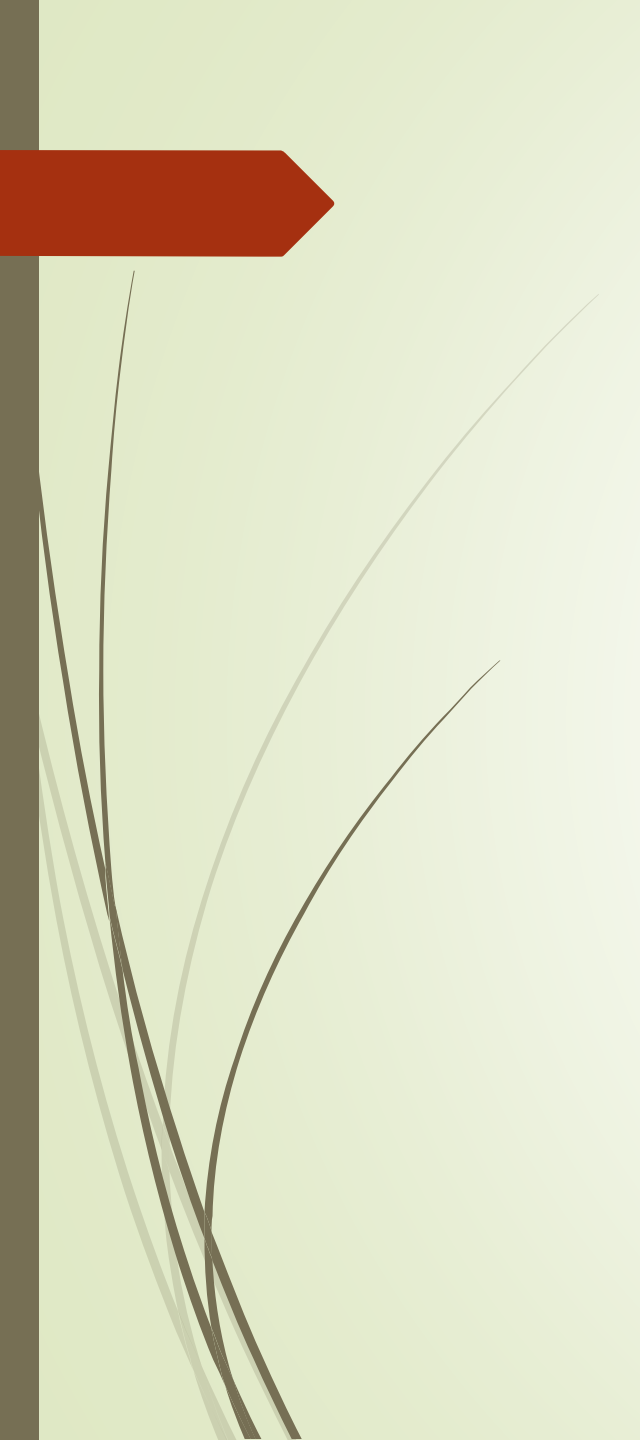
- ▶ Linked list nodes composed of two parts
 - ▶ Data part
 - ▶ Stores an element of the list
 - ▶ Next part
 - ▶ Stores link/pointer to next element
 - ▶ Stores Null value, when no next element





Simple Linked List Class

- We use a struct **Node** and class **List**
- Declare **Node** for the data points
 - data: int data in this example
 - next: a pointer to the next node in the list
- Declare **List**, which contains
 - head: a pointer to the first node in the list
 - Since the list is empty initially, head is set to NULL

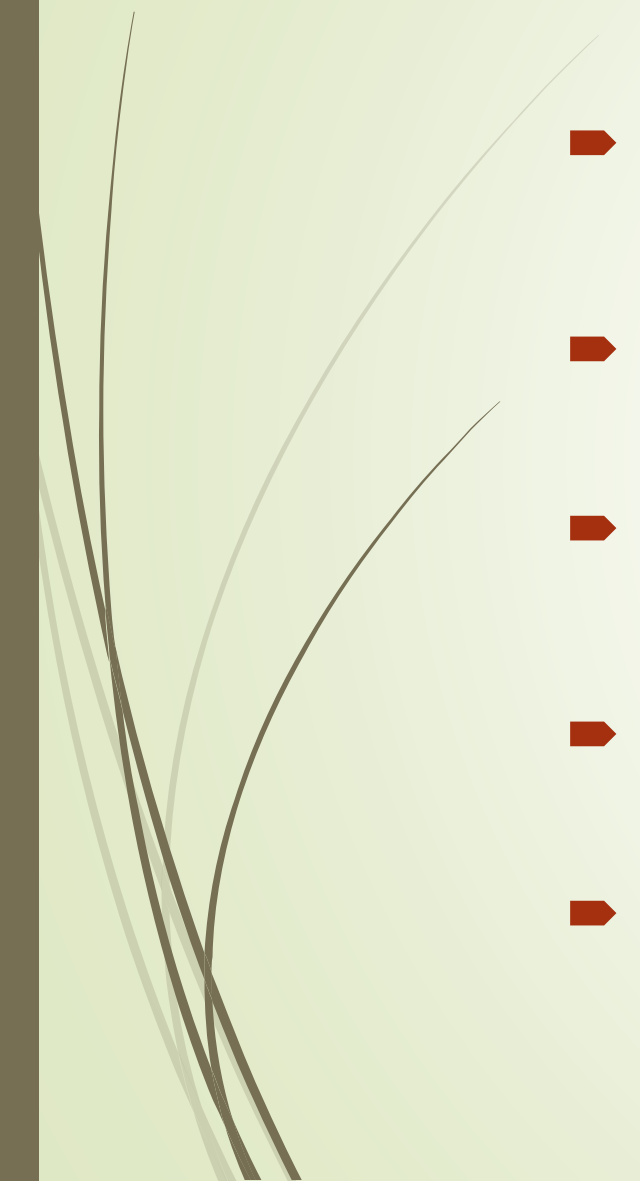


```
struct node{  
    int data;  
    node *next;  
};
```

```
class list{  
    private:  
  
    node *head;  
    public:  
        list()  
        {  
            head=NULL;  
        }  
};
```

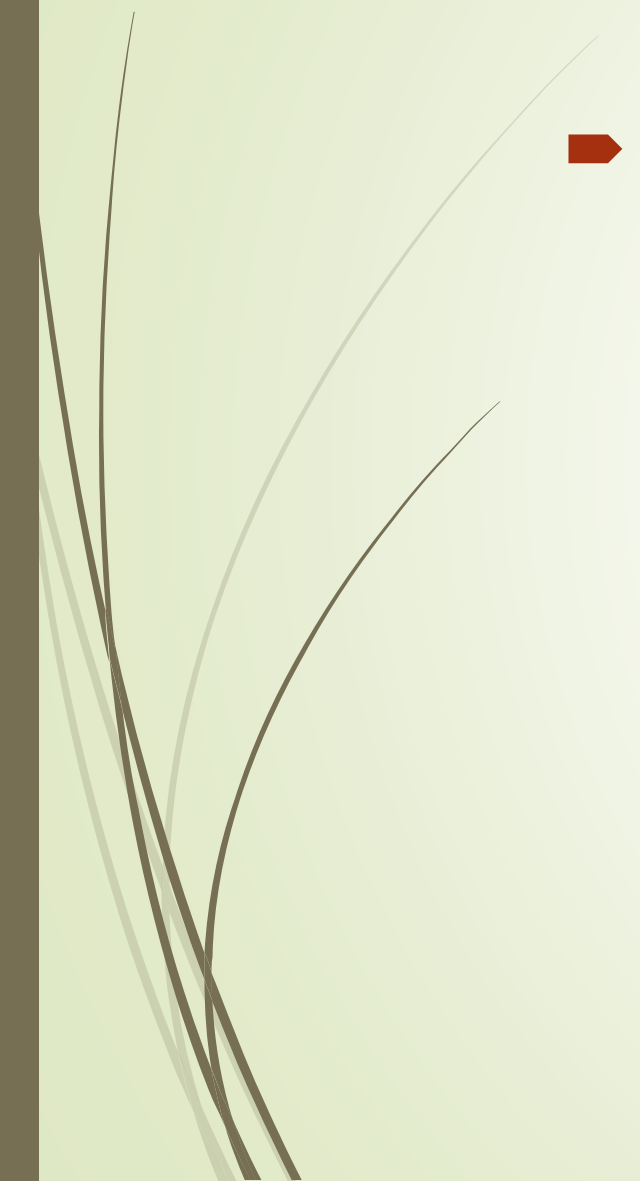


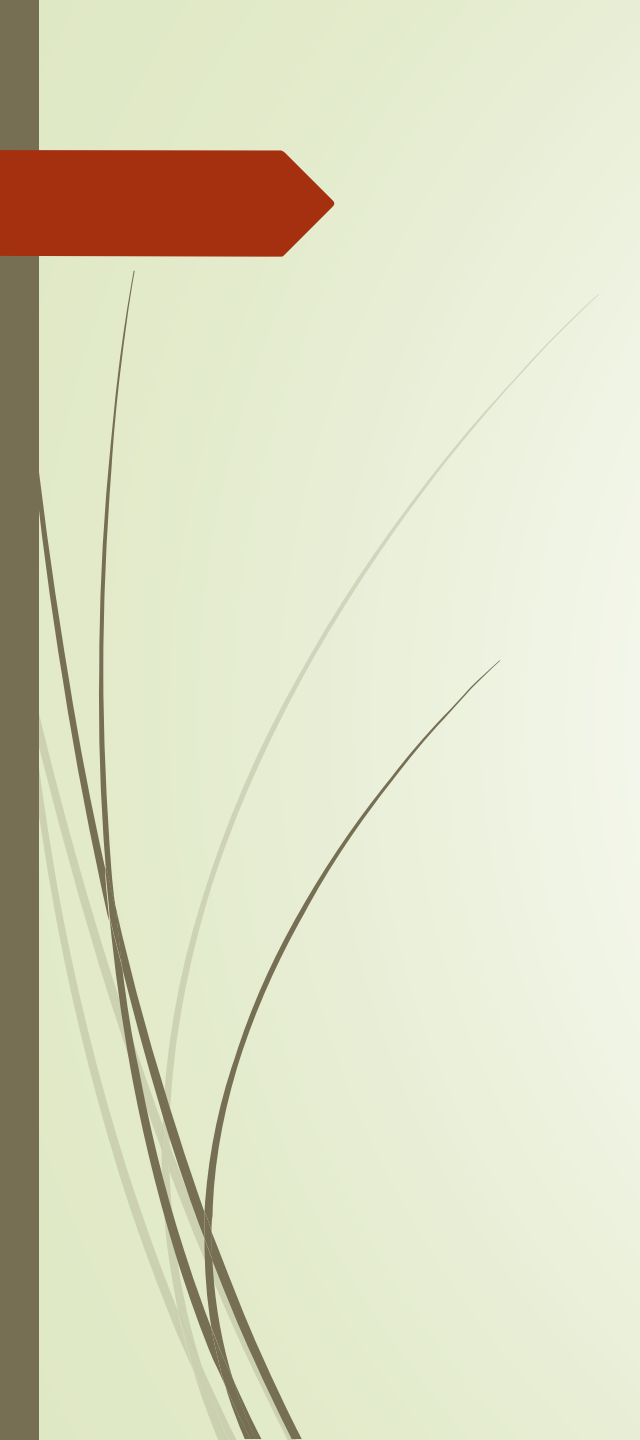
Operations of List

- IsEmpty: determine whether or not the list is empty
 - InsertNode: insert a new node at a particular position
 - FindNode: find a node with a given value
 - DeleteNode: delete a node with a given value
 - DisplayList: print all the nodes in the list
- 



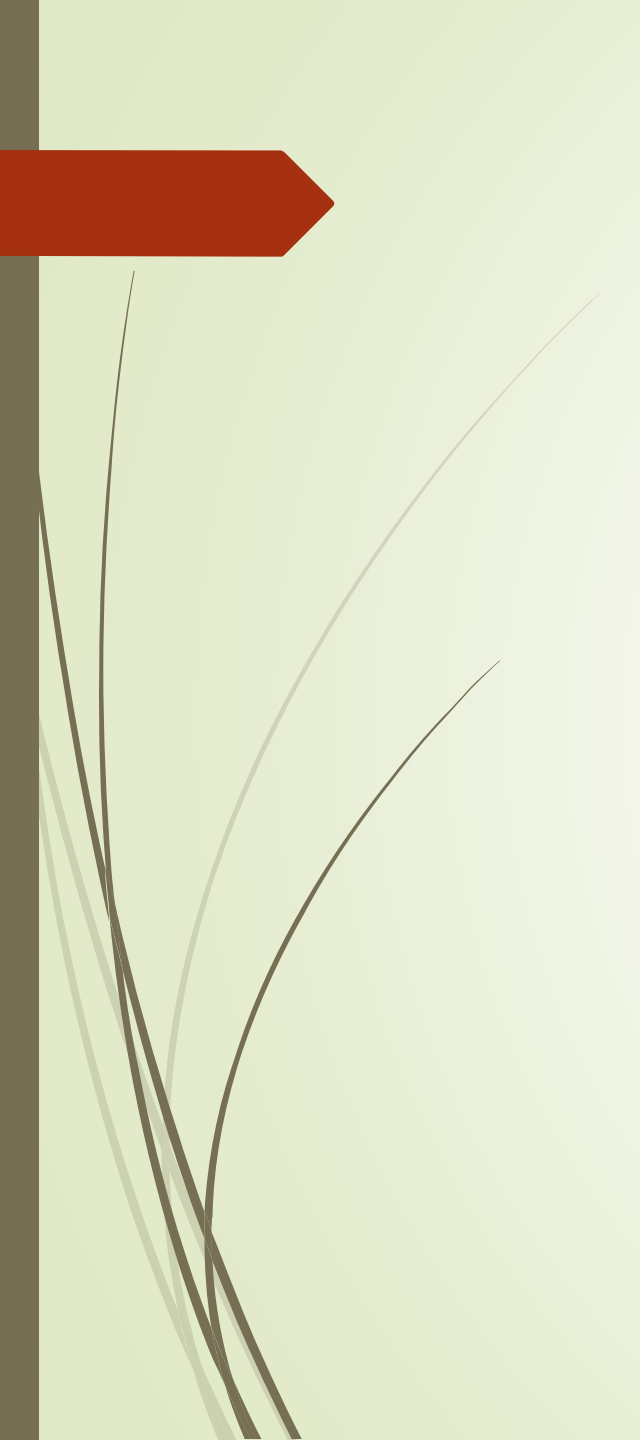
Inserting a Node

- Four Insertions
 - At start
 - At End
 - Before/After a Value
 - Before/After a Location
- 



```
void insert_at_start(int v)
{
    node *temp=new node;
    temp->data=v;
    temp->next=NULL;

    if(head==NULL)
    {
        head=temp;
    }
    else
    {
        temp->next=head;
        head=temp;
    }
    temp=NULL;
    delete temp;
}
```

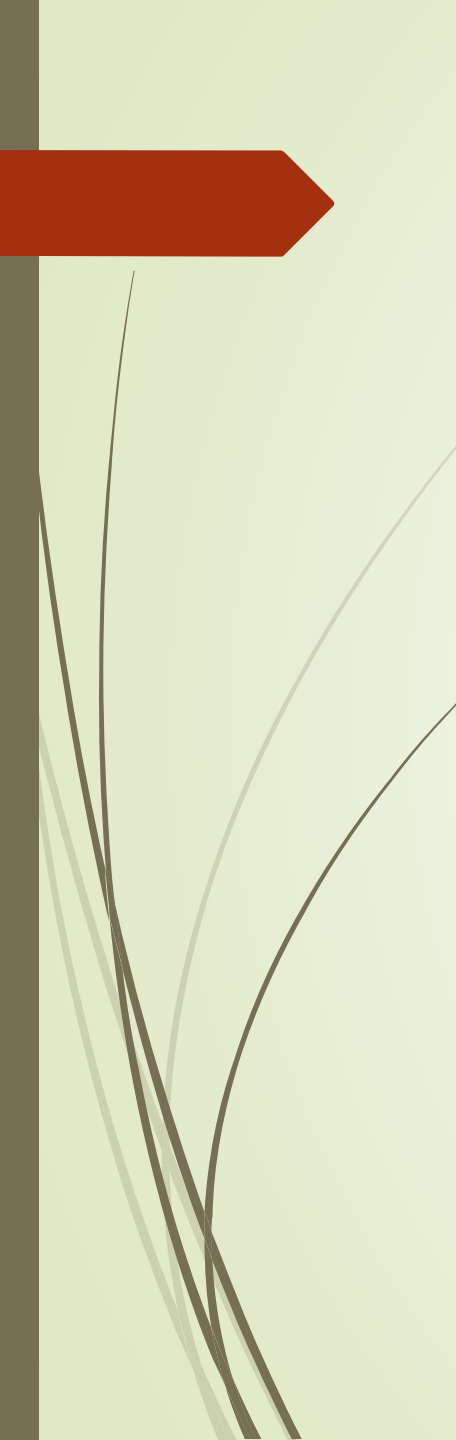


```
void insert_to_end(int v)
{
    node *temp=new node;
    temp->data=v;
    temp->next=NULL;

    if(head==NULL)
    {
        head=temp;
    }
    else
    {
        node *p=head;
        while(p->next!=NULL)
        {
            p=p->next;
        }
        p->next=temp;
    }
    temp=NULL;
    delete temp;
}
```



```
void insert_at_loc(int l,int v)
{
    node *temp=new node;
    temp->data=v;
    temp->next=NULL;
    node *p=head,*q=head->next;
    for(int i=1;i<l-1;i++)
    {
        p=q;
        q=q->next;
    }
    p->next=temp;
    temp->next=q;
}
```



```
void insert_after_value(int c, int v)
{
    node *temp=new node;
    temp->data=v;
    temp->next=NULL;
    node *p=head,*q;
    while(p!=NULL&& p->data!=c)
    {
        p=p->next;
    }
    q=p->next;
    p->next=temp;
    temp->next=q;
    temp=NULL;
    delete temp;
}
```



Deleting a Node

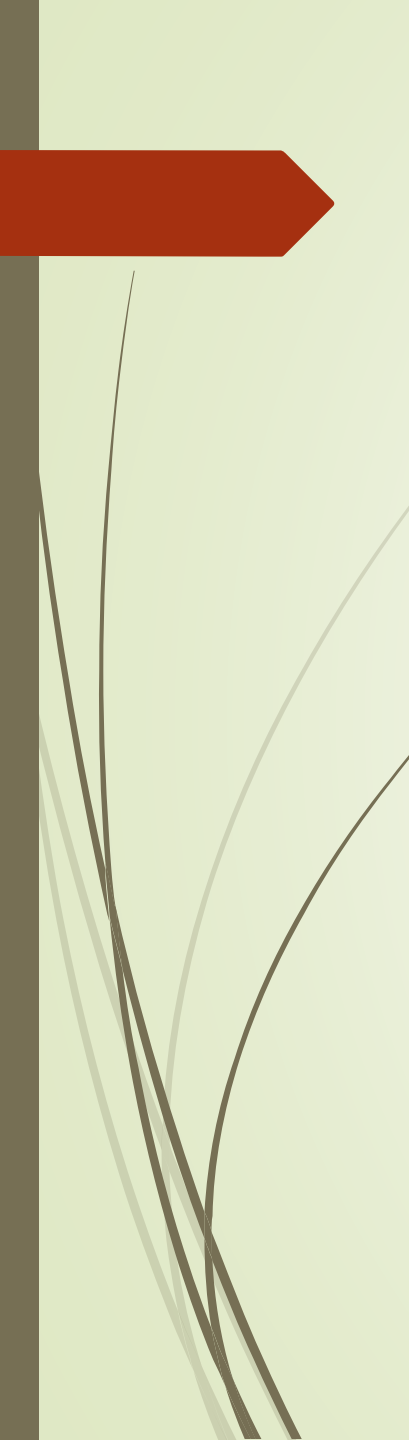
- Four Deletions

- At start

- At End

- A Value

- A Location

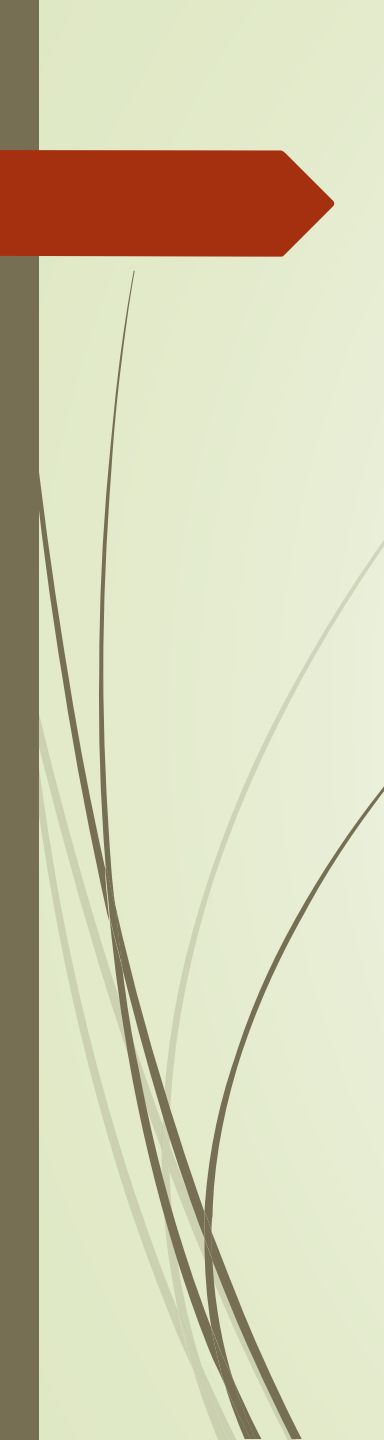


```
void delete_at_start()  
{  
    node *q=head;  
    head=head->next;  
    delete q;  
}
```



```
void delete_at_end()
{
    node *pre=head,*curr=head;
    while(curr->next!=NULL)
    {
        pre=curr;
        curr=curr->next;
    }

    pre->next=NULL;
    delete curr;
    curr=NULL;
}
```



```
void delete_a_location(int l)
{
    node *pre=head,*curr=head;
    for(int i=1;i<l;i++)
    {
        pre=curr;
        curr=curr->next;
    }
    pre->next=curr->next;
    delete curr;
    curr=NULL;
}
```

```
void delete_a_value(int v)
{
    node *pre=head,*curr=head;
    while(curr->data!=v&&curr->next!=NULL)
    {
        pre=curr;
        curr=curr->next;
    }
    pre->next=curr->next;
    delete curr;
    curr=NULL;
}
```

Search a Node

```
void search(int v)
{
    node *p=head;
    int c=0;
    while(p!=NULL)
    {
        c++;
        if(p->data==v)
            break;
        p=p->next;
    }
    if(p!=NULL)
        cout<<"datafoundat"<<c;
}
```

Printing a List

```
void display()
{
    if(head==NULL)
        cout<<"Empty";
    node *p=head;
    while(p!=NULL)
    {
        cout<<p->data<<" ";
        p=p->next;
    }
    cout<<endl;
}
```

Using a List

```
int main()
{
    list l;
    l.insert_at_start(77);
    l.insert_at_start(1);
    l.insert_at_start(3);
    l.insert_at_start(2);
    l.insert_at_start(5);
    l.insert_at_start(7);
    l.insert_at_start(9);
    l.insert_at_start(28);
    l.insert_to_end(34);
    l.insert_at_loc(4,55);
    l.insert_after_value(3,65);
    l.display();
    l.search(65);
    l.delete_at_start();
    l.delete_at_end();
    l.delete_a_location(4);
    l.delete_a_value(7);
    l.display();
}
```



```
28 9 7 55 5 2 3 65 1 77 34
```

```
data found at:8
```

```
9 55 2 3 65 1 77
```
